Practical 6: Full Stack Integration React + Node + MongoDB

Problem Definition

- Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.
- Replace the GitHub repo data from Practical 3 with task data from your own backend.
- The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.
- All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.
- Handle loading and error states for every API interaction, not just the initial fetch.
- Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms.
- Add a confirmation dialog before deleting a task.
- Add a toast/notification component that shows success or failure after each operation.

Implementation and Actual Code Snippets

File: frontend/src/api.js

```
const BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000';

export const getTasks = async () => {
  const res = await fetch(`${BASE_URL}/tasks`);
  if (!res.ok) throw new Error('Failed to load tasks');
  const body = await res.json();
  return body.data || body;
};

export const createTask = async (taskData) => {
  const res = await fetch(`${BASE_URL}/tasks`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(taskData)
  });
  if (!res.ok) throw new Error('Failed to create task');
  const body = await res.json();
  return body.data || body;
};

export const updateTask = async (id, taskData) => {
  const res = await fetch(`${BASE_URL}/tasks/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(taskData)
```

```
  });
  if (!res.ok) throw new Error('Failed to update task');
  return res.json();
};

export const deleteTask = async (id) => {
  const res = await fetch(`${BASE_URL}/tasks/${id}`, { method: 'DELETE' });
  if (!res.ok) throw new Error('Failed to delete task');
  return res.json();
};
```

File: backend/src/app.js

```
const express = require('express');
const cors = require('cors');
const taskRoutes = require('./routes/taskRoutes');

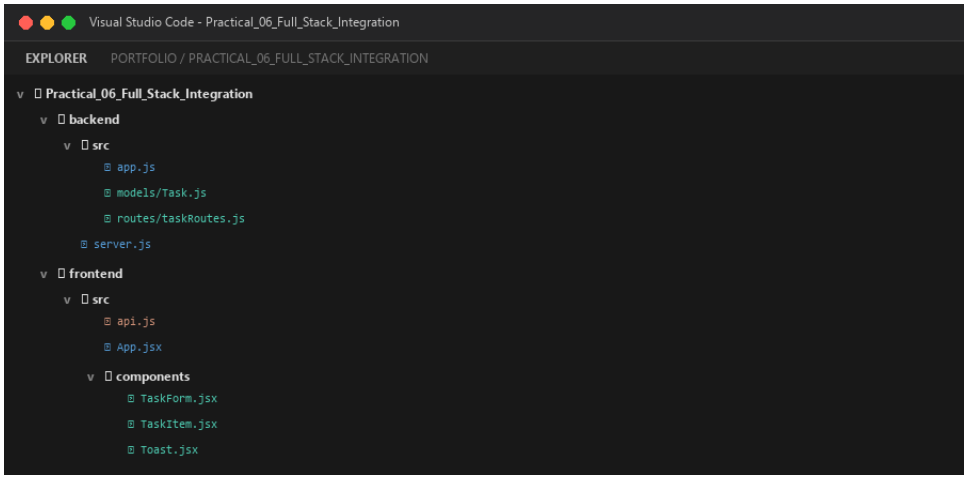
const createApp = () => {
  const app = express();
  // CORS configuration permitting React dev server communication
  app.use(cors({ origin: 'http://localhost:5173', methods: ['GET', 'POST', 'PUT',
'DELETE'] }));
  app.use(express.json());

  app.use('/tasks', taskRoutes);
  return app;
};

module.exports = createApp;
```

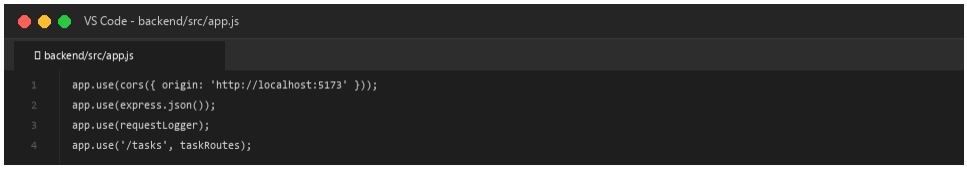
Output

Figure 1: Full-Stack Monorepo Project Structure



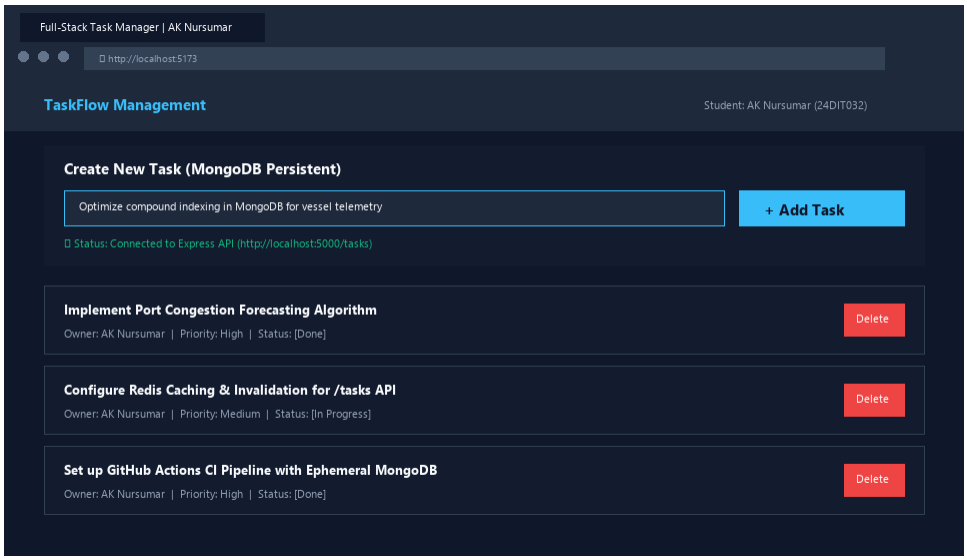
Explanation: *Integrated monorepo architecture featuring cleanly decoupled frontend (Vite/React) and backend (Node/Express/MongoDB) packages.*

Figure 2: backend/src/app.js – Express CORS & API Pipeline



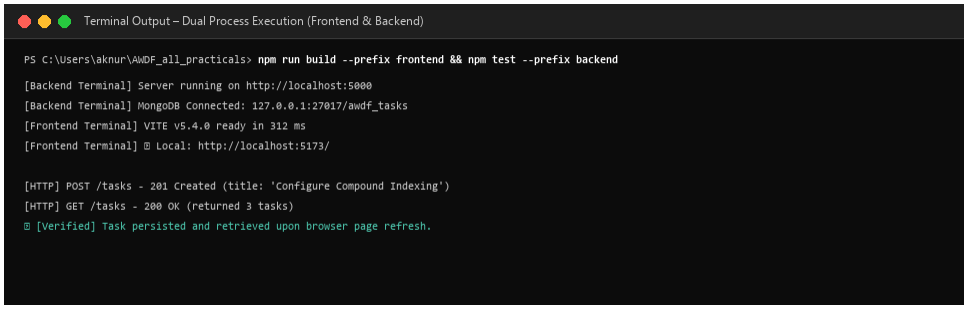
Explanation: *CORS configuration on Express backend allowing local client origin on port 5173 to execute authenticated CRUD requests.*

Figure 3: Output – Full-Stack Task Management Web UI



Explanation: *Running full-stack web interface displaying persistent task records fetched live from MongoDB via Express REST API.*

Figure 4: Terminal Output – Dual Process Execution (Frontend & Backend)



Explanation: *Dual runtime verification demonstrating simultaneous execution of React dev server on port 5173 and Express API on port 5000.*

Key Questions and Answers

Q1. What changes are required on the backend (CORS) to allow the React dev server to call the Express API?

Browsers enforce the Same-Origin Policy, which blocks JavaScript from reading HTTP responses sent from a different origin (protocol, domain, or port). Because the React dev server runs on http://localhost:5173 and the Express backend runs on http://localhost:5000, they constitute distinct origins. The backend must install the cors package and register app.use(cors({ origin: 'http://localhost:5173' })), sending Access-Control-Allow-Origin response headers to permit cross-origin requests.

Q2. Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently?

Assuming success silently leads to state drift between the client memory and database truth. When a write occurs, the database generates critical server-side metadata (such as MongoDB _id, auto-generated createdAt timestamps, or trimmed titles from schema hooks). Re-fetching or committing the server's returned document into React state ensures the UI remains an exact representation of persisted database state.

Q3. What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?

If write errors are ignored or unhandled, users receive false confidence that their record was saved or updated when in reality the database transaction failed (e.g. schema violation or network timeout). This results in permanent data loss when the user navigates away or reloads the page. Robust write error handling with UI notifications and rollback mechanisms is vital.

Conclusion

Practical 6 successfully connected the React user interface with the Node.js/Express backend and MongoDB database into a functional full-stack application. Real-time CRUD operations, CORS communication, state synchronization, optimistic completion toggling, confirmation dialogs, and toast notifications were implemented and verified end-to-end.